

Construction Robot Path-Planning for Earthwork Operations

Sung-Keun Kim, A.M.ASCE¹; Jeffrey S. Russell, M.ASCE²; and Kyo-Jin Koo, A.M.ASCE³

Abstract: In developing an intelligent mobile construction robot, a navigation system that can provide an effective and efficient path-planning algorithm is a very important element. The purpose of a path-planning method for a mobile construction robot is to find a continuous collision-free path from the initial position of the construction robot to its target position. This paper presents an improved Bug-based algorithm, called *SensBug*, which can produce an effective path in an unknown environment with both stationary and movable obstacles. The contributions, which make it possible to generate an effective and short path, are an improved method to select local directions, a reverse mode, and a simple leaving condition. Some emerging technologies that can be used for implementing an intelligent construction robot are introduced in this paper.

DOI: 10.1061/(ASCE)0887-3801(2003)17:2(97)

CE Database subject headings: Construction planning; Automation; Earthwork; Robotics.

Introduction

Recently, there has been an increase in demands to enhance intelligence of automation systems for construction operations in hazardous work environments such as underwater, in chemically or radioactively contaminated areas, and in regions with harsh temperatures. An automated construction system consists of several pieces of hardware (mobile robots) and software. In developing an intelligent mobile construction robot, a navigation system that can produce an effective and efficient path-planning algorithm is a very important element. The purpose of a path-planning method for a mobile construction robot is to find a continuous collision-free path from the initial position of the construction robot to its target position.

The research on robot path planning can be categorized into two models, based on different assumptions about the information available for planning: (1) path planning with complete information; and (2) path planning with incomplete information. The first model assumes that a robot has perfect information about itself and its environment. Information that fully describes the sizes, shapes, positions, and orientations of all obstacles in two-dimensional (2D) or three-dimensional (3D) space is known. Because full information is assumed, the path planning is a one-time and off-line operation (Lumelsky and Stepanov 1987; Lumelsky and Skewis 1988).

In the second model, an element of uncertainty is present, and the missing data is typically provided in real time by some source of local information through sensory feedback using an ultra-sound range or a vision module (Lumelsky and Skewis 1988). A robot has no information on its environment except a start position and a target position. The sensory information is used to build a global model for path planning in real time. The path planning is a continuous on-line process. The concept of path-planning algorithms in this category mostly comes from Lumelsky. The previous algorithms include Bug1 and Bug2 (Lumelsky and Stepanov 1987); VisBug1 and VisBug2 (Lumelsky and Skewis 1988); Angulus (Lumelsky and Tiwari 1994); DistBug (Kamon and Rivlin 1997); Tangent (Lee et al. 1997); and Tangent2 and CAT (Lee 2000). Construction and maintenance of a global model based on sensory information requires heavy computation, which is a burden on the robot (Kamon and Rivlin 1997). In reality, a mobile construction robot cannot have perfect information on its environment. The information about the location of obstacles should be collected by sensors on the construction robot, and a path can be generated based on the sensory information. This information can be obtained with visual, laser, ultrasonic, or photoelectric sensors. The theory and application of sensors for mobile robots are described in Everett (1995).

Kamon and Rivlin (1997) state that the performance of algorithms can be measured by two evaluation criteria: (1) total path length, and (2) path safety. Path safety should be considered while evaluating path quality. They suggest that the minimal distance between the robot and the surrounding obstacles from every location along the path can be used for measuring path safety. The bigger the average distance, the safer the path. However, it is difficult to do a direct comparison between algorithms, because the level of performance changes based on the different environments.

This paper presents an improved Bug-based algorithm, called *SensBug*, which can produce an effective path in an environment with both stationary and movable obstacles.

Shortcomings of Previous Bug-Based Path Planning Algorithms

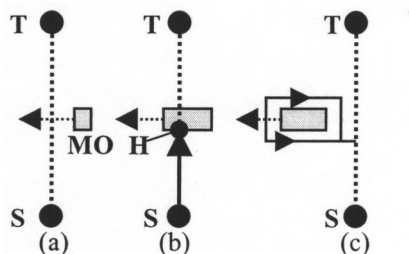
Most of the previous Bug-based path-planning algorithms generate the next path for a mobile robot in the environment only with

¹Senior Researcher, Korea Institute of Construction Technology, 2311 Taewha-Dong, Ilsan-Gu, Koyang-Shi, Kyunggi-Do, 411-712, South Korea. E-mail: skim68@kict.re.kr

²Professor, Dept. of Civil and Environmental Engineering, Univ. of Wisconsin-Madison, 2304 Engineering Hall, 1415 Engineering Dr., Madison, WI 53706. E-mail: russell@engr.wisc.edu

³Assistant Professor, The Univ. of Seoul, Dept. of Architectural Engineering, Cheonnong-Dong 90, Tongdaemoo, Seoul, 130-743, South Korea. E-mail: kook@uos.ac.kr

Note. Discussion open until September 1, 2003. Separate discussions must be submitted for individual papers. To extend the closing date by one month, a written request must be filed with the ASCE Managing Editor. The manuscript for this paper was submitted for review and possible publication on August 31, 2001; approved on September 10, 2002. This paper is part of the *Journal of Computing in Civil Engineering*, Vol. 17, No. 2, April 1, 2003. ©ASCE, ISSN 0887-3801/2003/2-97-104/\$18.00.



Note- T: Target Point, S: Start Point, H: Hit Point, and MO: Movable Obstacle

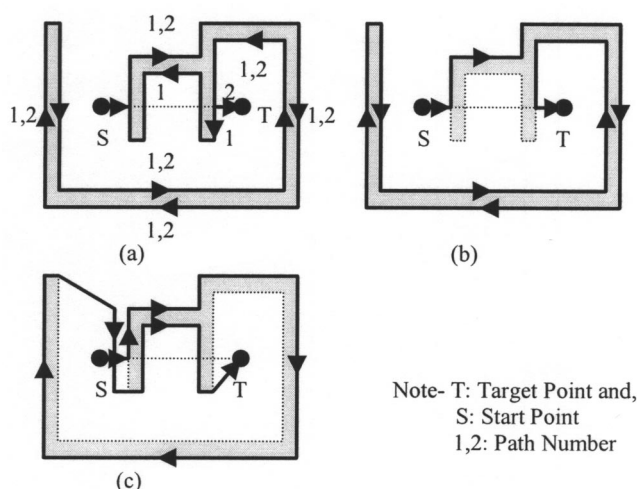
Fig. 1. Problem of previous bug-based path planning [modified from Lumelsky and Harinarayan (1997)]

stationary obstacles; thus, they may fail to produce an effective path in an environment with both stationary and movable obstacles. While traveling from S to T, a robot encounters a movable object at H (Fig. 1). The robot moves along the boundary of the movable object in a local direction (left) that is a default set by the path-planning algorithm. While the robot moves along the obstacle's boundary, the object also moves. The robot cannot leave the boundary of the obstacle, because the leaving condition described in the path-planning algorithms cannot be satisfied (Lumelsky and Harinarayan 1997).

The previous path-planning algorithms produce a relatively long path to arrive at their target in an unknown environment with complex stationary or movable obstacles. Whenever a robot encounters an obstacle, it makes a left turn by default; thus, it is hard for the robot to generate an effective path to reach the target. Some examples are shown in Fig. 2.

Improved Bug-Based Path Planning Algorithm

This section focuses on the development of a path-planning algorithm in an environment with multiple stationary and movable obstacles. The proposed algorithm is an extended version of the Bug algorithm developed by Lumelsky and Skewis (1988).



Note- T: Target Point and, S: Start Point
1,2: Path Number

Fig. 2. Path generation of previous bug-based algorithms: (a) Bug 1; (b) Bug 2; (c) Dist Bug

Assumptions

The assumptions are divided into two parts: (1) the geometry of the environment; and (2) the characteristics and capabilities of a mobile construction robot (Lumelsky and Skewis 1988).

Environment Assumptions

The environment is a 2D plane and is occupied by (1) stationary obstacles such as construction material and products, and (2) movable obstacles such as other mobile construction robots. The environment includes a Start (S) and a Target (T) point. The obstacles are simple curves of finite length. It is assumed that there are only a finite number of obstacles in the construction site and that the obstacles do not touch each other.

Mobile Construction Robot Assumptions

It is assumed that a mobile construction robot is a point, which means any distinct obstacles can be passable by the mobile construction robot. The mobile construction robot is given the coordinates of the start, the target, and its current position by using global positioning system (GPS) technology. Thus, it can always calculate its direction and its distance from the target. It also has communication ability with other mobile construction robots to send and receive information on positions and environment. When the mobile construction robot senses obstacles, it can distinguish movable obstacles from stationary obstacles in the environment. If a sensed obstacle is another mobile construction robot, then the sensed construction robot can be set as a *Master* or a *Slave* through communication. A dynamic path for the Master will be generated by the path-planning algorithm to avoid the Slave, and the Slave makes just a straight path. The mobile robot has a memory to store position data and intermediate results and is equipped with a range sensor, displacement sensors, a GPS receiver, and a communicator. In reality, the sensing range can be larger than 200 m with ± 5 cm of measuring accuracy. There are four possible action modes: (1) move toward the Target; (2) move toward the hit point; (3) move along the obstacle boundary; and (4) stop.

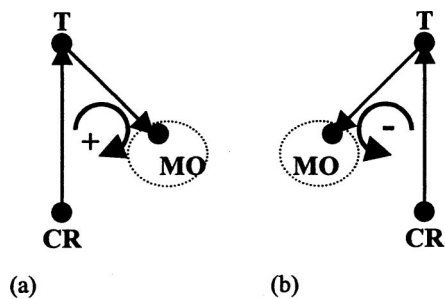
Notations and Definitions

Throughout this paper, the following notation is used: (1) *M-line*, called a main line, is a straight line from Start to Target; and (2) *M'-line*, called a current main line, is a straight line from the current position of the mobile construction robot to the Target.

Definition 1. A *local direction* is a decided direction for moving around the obstacle boundary. For the a two-dimensional area, it can be either left or right (Lumelsky and Stepanov 1987).

Definition 2. If three points, consisting of (1) the current point of a mobile construction robot, (2) the point of a target, and (3) the current point of a movable obstacle, form a clockwise circuit when viewed from the mobile construction robot, the movable obstacle is a *positive obstacle*. Otherwise, it is a *negative obstacle* (Fig. 3).

Definition 3. If a mobile robot encounters an obstacle while moving along a straight line toward a target, the point on the obstacle's boundary where the mobile robot begins to move along the boundary is a *hit point* (Lumelsky and Stepanov 1987). When the mobile robot leaves the obstacle at a point in order to continue its navigation to the target, the point is a *leave point* (Lumelsky and Stepanov 1987).



Note- T: Target, MO: Movable Obstacle, and CR: Construction Robot

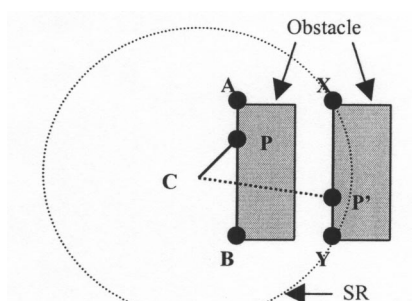
Fig. 3. Concept of (a) positive and (b) negative obstacles

Definition 4. A point P on an obstacle boundary is a *sensible point* if and only if the closed line segment between the current position of the mobile construction robot and the point P is not interrupted, and P is within the sensing range of the mobile construction robot. This concept is similar to a *visible point* as described in Lumelsky and Skewis (1990).

In Fig. 4, if $P \in \overline{AB}$, then P is sensible, because $\overline{CP}$ is not interrupted by the obstacle. However, if $P' \in \overline{XY}$, then P' is not sensible, because $\overline{CP'}$ is interrupted by the obstacle.

Selection of Start and Target Positions

The construction site for earthwork operations can be partitioned into several quadrangle areas using a quadtree data structure, which has resign information in a four-way branching tree and consists of $2^n \times 2^n$ leaf nodes (Samet 1990), as shown in Fig. 5. The leaf node represents one of the work areas on the construction site. After partitioning the construction site, the cut and fill volumes for each leaf node are calculated. The volume calculation is easily achieved by surface-to-surface comparison with cut and fill contours or surface to a planned plane. There are three available types of leaf nodes. One is the source node whose cut volume is larger than its fill volume, which means that it can provide other nodes with soil. Another is the target node whose fill volume is larger than its cut volume, which means that it needs soil to be filled, spread, and compacted. The other is the source-target node whose cut volume is equal to fill volume or whose cut and fill volumes are zero, which means that the stripped soil of a leaf node is moved from one portion of the node to another portion of it or the earthwork operation is not needed.



Note - C: Current position of a mobile construction robot and SR: Sensing range

Fig. 4. Sensible and insensible points

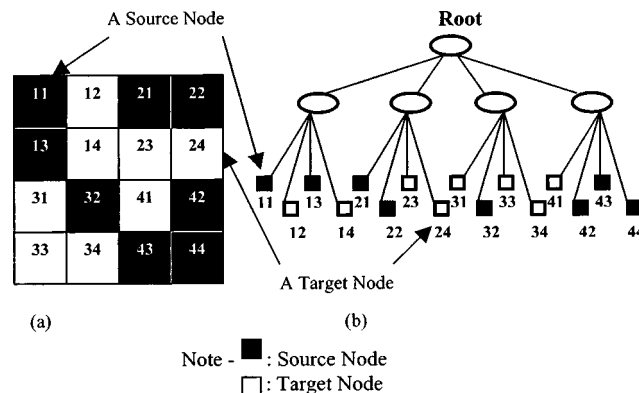


Fig. 5. Example of construction site partition: (a) construction site; (b) quadtree data structure

To transport stripped soil to a target area, one of the points in a source or a source-target node can be a start point, and one of the points in a target or a source-target node can be a target point for a mobile construction robot. To return to the source area, vice versa, one of the points in the target node can be a start point and one of the points in the source node can be a target point for the robot.

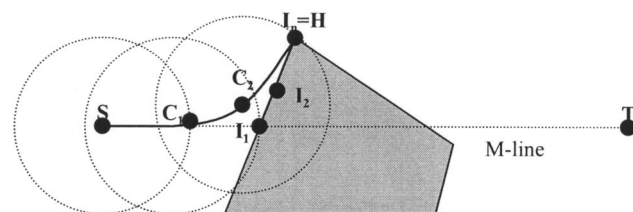
SensBug Algorithm

The *SensBug* algorithm generates a path for a mobile construction robot in a 2D planar unknown environment populated by stationary and movable obstacles. The *SensBug* algorithm uses four basic action modes: (1) move toward the Target; (2) move toward the hit point mode; (3) move along the boundary of an obstacle; and (4) stop.

Initially, the mobile construction robot moves directly toward the Target. If an obstacle is detected, then it checks the type of the obstacle (stationary or movable obstacle). If the detected obstacle is movable, then it checks the type of the movable obstacle (positive or negative). After these steps, it moves to a hit point on the obstacle's boundary and switches to the obstacle's boundary following mode. During boundary following, the mobile construction robot records all leave points. When it leaves the obstacle, it moves toward to the Target.

Hit Point Selection

In Fig. 6, if an obstacle is identified crossing the M-line at the point C_1 , a sensible intermediate point (I_1), which is an intersection point of the M-line and obstacle boundary, can be found. I_1 is considered an intermediate target at this moment. As the mobile



Note - S: Start point, C: Current point, I_i : Intermediate point, H: Hit point, and T: Target. ($i = 1, 2, \dots, n$)

Fig. 6. Example of hit point

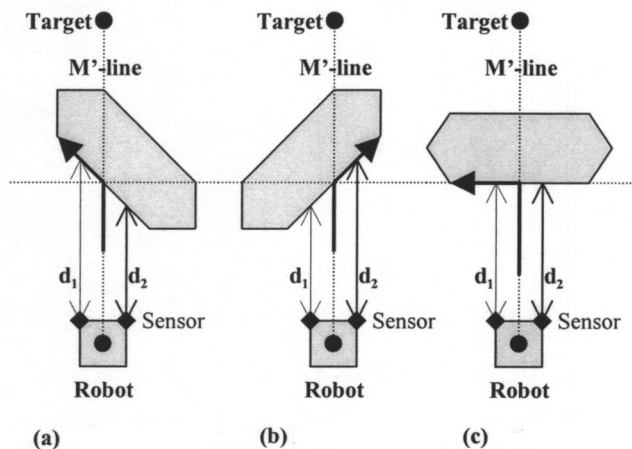


Fig. 7. Local direction for stationary obstacles: (a) left: $d_1 > d_2$; (b) right: $d_1 < d_2$; (c) left: $d_1 = d_2$

construction robot moves toward I_1 , it senses another sensible intermediate point (e.g., I_2), which is where the obstacle boundary in the local direction intersects the maximum sensing range. A newly identified point should construct a continuous segment from I_1 along the obstacle boundary. The mobile construction robot changes the intermediate target and moves toward the point I_2 while sensing the next intermediate point in the local direction. The key is that the hit point selection is continued until the mobile construction robot cannot find a new intermediate point in the local direction. At the point C_2 , the intermediate point I_n is sensed. If the mobile construction robot passes the point C_2 , it can no longer detect a new intermediate point, and the point I_n is set as a hit point.

Local Direction

The local direction is determined based on the local information. A method for choosing the local direction, default direction, is described as follows:

1. Local direction for stationary obstacles (construction materials and products): When a mobile construction robot encounters this type of obstacle, as shown in Fig. 7, the noncontact displacement sensor on the mobile robot measures the distance between the current position of the robot and the obstacle's boundaries (the right side and the left side of the M' -line). The mobile robot makes a turn to the direction of

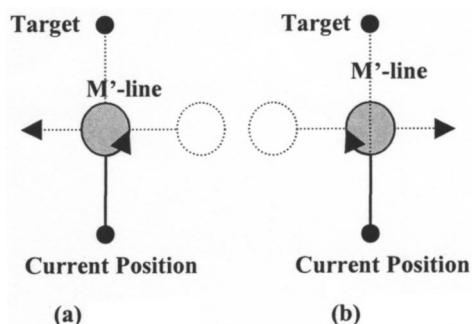


Fig. 8. Local direction for movable obstacles: (a) right; (b) left

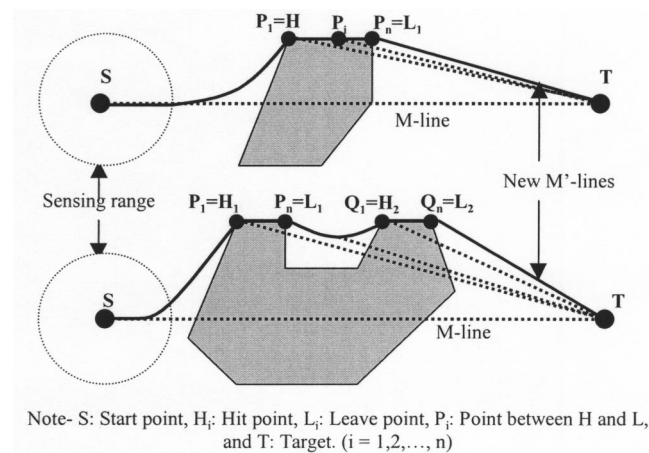


Fig. 9. Example of leave point selection (1)

the boundary, which is the far side from the current position of the robot (Kamon and Rivlin 1997). As a result, the robot can be closer to the target (Fig. 7).

2. Local direction for movable obstacles (other mobile construction equipment or robots): When a mobile construction robot, a Master, detects a movable obstacle, it checks whether the obstacle is a positive obstacle or not. If the obstacle is a positive one and is approaching the M' -line, then the local direction is right. If the obstacle is a negative one and is approaching the M' -line, then the local direction is left. This is illustrated in Fig. 8. If the detected movable obstacle is on the M' -line, which means it is neither a positive nor a negative one, then the default direction is left.

Reverse Mode

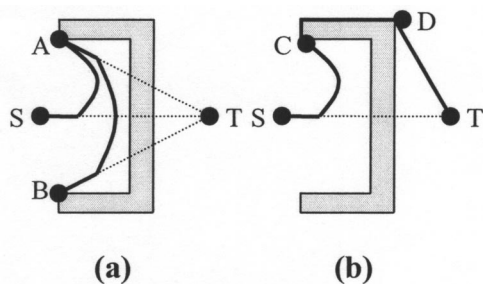
As mentioned earlier, the mobile construction robot has a memory to store all hit points and the default local direction at each hit point. If the mobile construction robot detects a hit point that it has already visited, then the local direction becomes the opposite direction of the default direction.

Leave Point Selection

After a mobile construction robot arrives at a hit point, it moves along the obstacle boundary while checking *movability*, which means that from the current position, the mobile construction robot can directly move along a newly identified M' -line. In Fig. 9, the mobile construction robot cannot move along a newly identified M' -line until it reaches the point P_n or Q_n , because the path is blocked by an obstacle. At the point P_n or Q_n , the mobile construction robot can move toward the target. P_n and Q_n are defined as leave points, L_1 and L_2 , respectively. An important thing is that the leave point selection is executed after moving along an obstacle boundary to overcome a concave obstacle, such as in Fig. 10(b). If the leave point selection can be executed without running the obstacle boundary following mode, then the mobile robot cannot overcome the concave obstacle [Fig. 10(a)]. Points A and B are leave points as well as hit points. At these points, the mobile robot can directly move toward the target, but it cannot deal successfully with the given obstacle. It goes back and forth between points A and B infinitely.

SensBug Algorithm

Step 1: *Move toward the target mode.* Move along the M -line or M' -line while scanning the environment until one of the following conditions is met:



Note- S: Start Point, T: Target, A, B, C: Hit Point, A, B: Wrong Leave Point, and D: Right Leave Point

Fig. 10. Example of leave point selection (2)

1. The Target is reached. Go to Step 4.
2. An obstacle that is crossed by the M-line or M'-line is detected. Go to Step 2.

Step 2: *Move toward the hit point* mode. Determine the type of the obstacle and the local direction through communication. Move until the hit point on the obstacle boundary is reached. Go to Step 3.

Step 3: *Move along the obstacle boundary* mode. Follow the obstacle boundary until one of the following events occurs:

1. The target is sensible. Go to Step 1.
2. The mobile construction robot senses the same hit point three times. Go to Step 4.
3. Leaving condition holds. Go to Step 1.

Step 4: *Stop* mode. The procedure is ended.

Theorem 1. The *SensBug* algorithm always terminates if and only if the number of obstacles is finite.

Proof. If the mobile construction robot encounters an obstacle, the *SensBug* algorithm changes its mode from *Move toward the target* to *Move toward hit point*. After the mobile construction robot arrives at a hit point, it moves along the obstacle boundary. From this moment, there are three possibilities: (1) If the target can be sensed, the mobile construction robot can leave the obstacle boundary and directly reach the target, and then the *SensBug* algorithm terminates; (2) if the leaving condition holds, the mobile construction robot leaves the obstacle boundary and moves directly along a newly identified M'-line, and then the *Move toward the target* mode is activated; or (3) if the mobile construction robot is not able to leave the obstacle boundary, the mobile construction robot will sense the hit point it has already visited twice, and the *SensBug* algorithm terminates after following a finite length path.

Theorem 2. Using the *SensBug* algorithm, a mobile construction robot can reach the target if and only if the target is reachable from the start, the number of obstacles is finite, and the sensing and communication ability of the robot is available.

Proof. *Move toward the target* mode terminates at the target or a sensible intermediate target point that is an intersection point of the M'-line and an obstacle boundary; *Move toward the hit point* mode terminates at a hit point; and *Move along the obstacle boundary* mode terminates at a leave point where a mobile construction robot can move along a newly identified M'-line. If the number of obstacles is finite, then the number of *Move toward the*

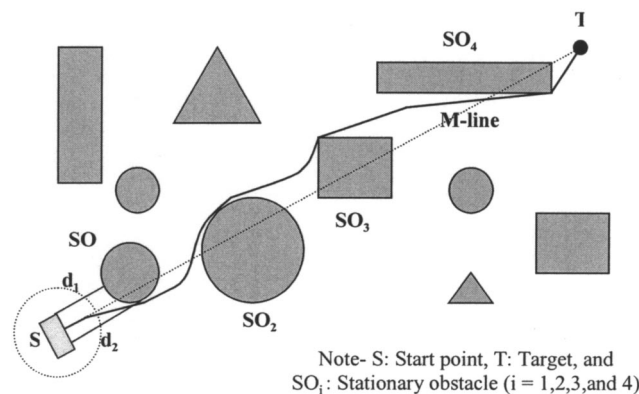


Fig. 11. Environment with stationary obstacles

hit point and *Move along the obstacle boundary* modes is also finite. If the robot reaches a hit point, then the *Move toward the hit point* mode is followed by the *Move along the obstacle boundary* mode. If the robot does not reach the target, then the *Move along the obstacle boundary* mode is followed by the *Move toward the target* mode. Since the *Move along the obstacle boundary* mode number is finite, the *Move toward the target* mode number is also finite. For this reason, there is the last chance of executing the *Move toward target* mode, which means that the robot can reach the target.

Example Simulations

To evaluate the *SensBug* algorithm, example simulations were performed in the Windows NT environment using an application implemented by the writers. For a stationary obstacle environment, Simulation Library (SL) Version 2.1 developed by Hert and Reznik (1997) is a good tool for simulations. The SL is a collection of C-language functions, which provide a framework in which to develop computer simulations of 2D robotic systems. It provides a high-level C-language interface, which can define a robot, its environment, and the interaction between them.

Case 1

Fig. 11 illustrates a path taken by *SensBug*. During navigation to the target, the mobile construction robot encounters four stationary obstacles. When it encounters SO_1 and SO_4 , its local direction is right, because d_2 is larger than d_1 . When it encounters SO_2 and SO_3 , its local direction is left, because d_1 is larger than d_2 . In an environment with stationary obstacles, the *SensBug* algorithm generates a shorter path than the other Bug-based algorithms.

Case 2

Fig. 12(a) shows an initial environment configuration before movable obstacles move, and Fig. 12(b) illustrates the path generated by the *SensBug* algorithm in the movable obstacle environment. At points A, B, and C, the mobile construction robot senses a movable obstacle and moves in the default local direction to avoid collision. Previous Bug-based algorithms cannot generate an effective path in a movable obstacle environment. Due to the variable local direction, the reverse mode, and the simple leaving

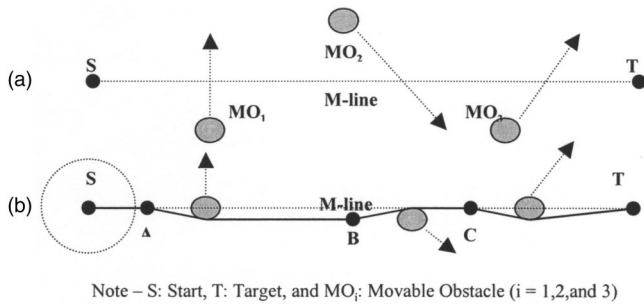


Fig. 12. Environment with movable obstacles

condition, the *SensBug* algorithm can overcome the shortcomings of existing path planning algorithms and can generate a relatively shorter path than other path-planning algorithms.

Case 3

Fig. 13 shows a path by *SensBug* in a complex stationary environment. The target is located inside of the stationary obstacle. The mobile construction robot moves toward the target in the local direction (left). When it arrives at the hit point A, the path for the mobile construction robot is generated by the *Move along the obstacle boundary* mode. From the point B, which is a leave point, the *SensBug* algorithm starts the *Move toward the target* mode. When it arrives at point C, it senses the hit point A that it has already sensed and visited before. Thus, the default local direction is changed (right) and the mobile construction robot reaches point D, the new hit point. At point E, the leave condition is satisfied. The mobile construction robot leaves the concave obstacle and arrives at T.

Case 4

Fig. 14(a) shows an initial configuration before a movable obstacle moves. The mobile construction robot moves toward the target. It follows the M-line until an obstacle is detected. When the mobile construction robot senses SO_1 at point A, the *SensBug* algorithm changes its mode to the *Move toward the hit point* mode. When it arrives at the hit point B, the path for the mobile construction robot is generated by the *Move along the obstacle boundary* mode. While moving from B to C, the mobile construction robot checks its movability. The leaving condition is met at point C, and thus the mobile construction robot leaves the obstacle boundary. With the same process as above, the path from C to D is generated to avoid the next obstacle, SO_2 . At point E, the

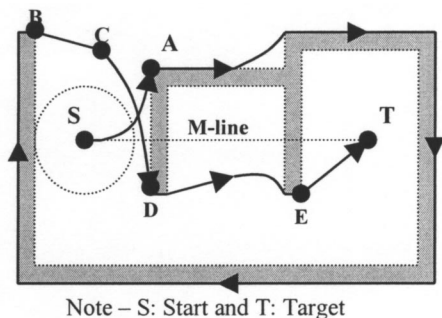


Fig. 13. Environment with complex stationary obstacle

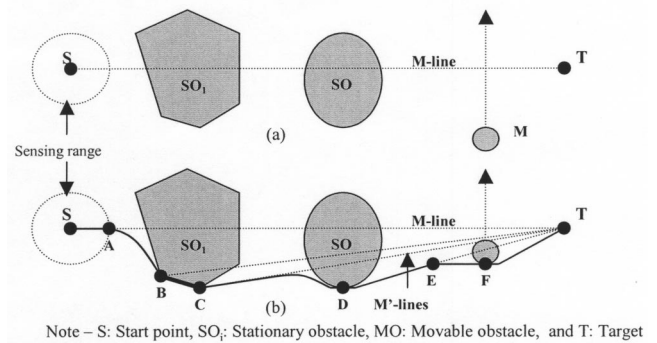


Fig. 14. Environment with stationary and movable obstacles

mobile construction robot senses a movable obstacle through communication and determines its local direction. The path is generated by the *Move toward the hit point* mode. After passing the point F, the algorithm resumes the *Move toward the target* mode and the mobile construction robot arrives at the target. The process is then ended.

Available Technologies for System Implementation

There are several emerging technologies that can be adapted to implement an automated navigation system for mobile construction robots. The system cannot be successful without efficient and real-time monitoring, controlling, and decision-making abilities. Other industries such as the mechanical and manufacturing industries are valuable sources of these technologies. Some technologies from other industries are not directly suitable for the construction industry, however, some modification is required to satisfy these needs. The technologies used for the automated navigation system are briefly reviewed in this section.

Distributed Artificial Intelligence (DAI)

DAI is a subfield of artificial intelligence (AI). It is concerned with solving problems by applying both artificial intelligence techniques and multiple problem solvers (Deker 1987). The world of DAI can be divided into two primary arenas: (1) distributed problem solving (DPS); and (2) multiagent system (MAS). Research in DPS considers how the work of solving a particular problem can be divided among a number of modules, or nodes, that cooperate at the level of dividing and sharing knowledge about the problem and development of a solution (Smith and Davis 1981). In MAS, research is concerned with coordinating intelligent behavior among a collection of autonomous robots and with how they can coordinate their knowledge, goals, skills, and plans jointly to take action or to solve problems.

There are several reasons why the DAI concept is appropriate for the automated navigation system. First, due to possible changes in the initial conditions for path planning, replanning to generate a new path is often necessary based on real-time data. Construction robot breakdowns, accidents, and other unexpected conditions are some factors that can cause changes to the initial path plan. DAI can provide an effective way to deal with these kinds of changes. Second, several construction robots that have distributed, heterogeneous functions are involved in a construction operation at the same time. They should perform tasks in a cooperative manner. DAI can provide insights and understand-

ing about interaction among construction robots on the construction site in order to solve problems. In addition, data from these robots should be interpreted and integrated. Third, every robot has a different capacity and capability. This implies that there are a great number of possible robot combinations that are time and cost effective to perform given tasks.

Global Positioning System (GPS)

The global positioning system (GPS) is a worldwide satellite-based navigation system operated and maintained by the U.S. Department of Defense. GPS provides several important features, including its high position accuracy and velocity determination in three dimensions, global coverage, all-weather capability, continuous availability to an unlimited number of users, accurate timing capability, ability to meet the needs of a broad spectrum of users, and jam resistance (Leick 1990). This technology can be used by mobile construction robots. A motion strategy for planning an efficient and exact path for the construction robot can be determined by using GPS position data with preplanned motion models. For the proposed system, several types of commercial GPS equipment are available. [e.g., GPS receiver (Trimble 7400 rsi), radio modem (TrimTalk 900), GPS antenna (Ruggedized L1/L2 antenna), and software (Trimble Total Control, Trimble Geomatics Office)].

Sensor and Sensing Technology

A sensor is a device or transducer that receives information about various physical effects such as mechanical, optical, electrical, acoustic, and magnetic effects and converts them into electrical signals. These electrical signals can be acted upon by the control unit (Warszawski and Sangrey 1985). The construction robot's ability to sense its environment and change its behavior based on sensory information is very important for an automated system. Without sensing ability, the construction robot is a common construction tool, just going through the same task again and again in a human-controlled environment. Such a construction tool is commonly used for construction operations, certainly has its place, and is often the right choice. With smart sensors, however, the construction robot has the potential to do much more. It can perform given tasks in unstructured environments and adapt as the environment changes around it. It can work in dirty and dangerous environments where humans cannot work safely. The sensor technologies are used for (1) real-time positioning, (2) real-time data collection during operation, (3) robot health monitoring, (4) work quality verification and remediation, (5) collision-free path planning, and (6) robot performance measurement.

Wireless Communication Technology

Wireless communication can be defined as a form of communication without using wires or fiber-optic cables over distance by the use of arbitrary codes. Information is transmitted in the form of radio spectra, not in the form of speech. Thus, information can be available to users at any time, in all places. The data transmitted can represent various types of information such as multivoice channels, full-motion video, and computer data (IBM 1995).

Wireless communication technology is very important for the automated navigation systems, because a construction robot moves from place to place on a construction site, and data and information needed should be exchanged between construction robots in real time. With wireless communication technology,

communication is not restricted by harsh construction environments due to remote data connection, and construction robots and human operators can expect and receive the delivery information and services no matter where they are on the construction site, or even around the construction site.

Conclusion

The purpose of a path-planning algorithm for a mobile construction robot is to find a continuous collision-free path from the initial position of the construction robot to its target position. This paper has suggested an improved Bug-based algorithm, called *SensBug*, for the navigation system of a mobile construction robot.

The *SensBug* algorithm generates a path for a mobile construction robot in a 2D planar unknown environment populated by stationary and movable obstacles. The mobile construction robot is equipped with a range sensor, displacement sensors, a GPS receiver, and a communicator. The *SensBug* algorithm uses four basic action modes: (1) move toward the Target; (2) move toward the hit point; (3) move along the boundary of an obstacle; and (4) stop.

The main improvements over existing methods are: (1) an improved method to decide a local direction, which allows the mobile construction robot to generate an effective path in the environment with both stationary and movable obstacles; (2) a reverse mode, which can provide a mobile construction robot with a way to overcome the problem of obstacles in a complex configuration; and (3) a simple leaving condition, which allows the mobile construction robot to leave the obstacle boundary as soon as possible. These improvements make it possible to overcome the weak points of the previous algorithms.

Some available technologies, such as DAI, GPS, sensor, and wireless communication technology, have been suggested to implement an intelligent mobile robot. If these technologies, which can provide a tool for real-time monitoring, controlling, and decision-making abilities, are not available, the navigation system cannot be successful.

Notation

The following symbols are used in this paper:

- C = current position of a mobile construction robot;
- CR = construction robot;
- d = distance between current point of robot and obstacle's boundary;
- H = hit point;
- I = intersection point of main-line and obstacle boundary;
- L = leave point;
- MO = movable obstacle;
- S = start position for mobile construction robot;
- SO = stationary obstacle;
- SR = sensing range; and
- T = target position for mobile construction robot.

Subscripts

- i, n = positive integer indices.

References

- Deker, K. (1987). "Distributed problem-solving techniques: A survey." *IEEE Trans. Syst. Man Cybern.*, 17(5), 729-740.

- Everett, H. R. (1995). *Sensors for mobile robots: Theory and application*, A. K. Peters, Wellesley, Mass.
- Hert, S., and Reznik, D. (1997). *The simulation library: A basis for animation programs, version 2.1*, University of Wisconsin-Madison, Madison, Wis.
- IBM International Technical Support Organization. (1995). *An introduction to wireless technology*, Research Triangle Park, N.C.
- Kamon, I., and Rivlin, E. (1997). "Sensory-based motion planning with global proofs." *IEEE Trans. Rob. Autom.*, 13(6), 814–812.
- Lee, S. (2000). "Spatial model and decentralized path planning for construction automation." PhD thesis, University of Wisconsin-Madison, Madison, Wis.
- Lee, S., Adams, T. M., and Byoo, B. (1997). "A fuzzy navigation system for mobile construction robot." *Autom. Constr.*, 6(2), 97–107.
- Leick, A. (1990). *GPS satellite surveying*, Wiley, New York.
- Lumelsky, V. J., and Harinarayan, K. R. (1997). "Decentralized motion planning for multiple mobile robots: the cocktail party model." *Auton. Robots J.*, 4, 121–135.
- Lumelsky, V. J., and Skewis, T. (1988). "A paradigm for incorporating vision in the robot navigation function." *Proc., IEEE Int. Conf. on Robotic Automation*, Institute of Electrical and Electronics Engineers, New York, 734–739.
- Lumelsky, V. J., and Stepanov, A. A. (1987). "Path-planning strategies for a point mobile automation moving amidst unknown obstacles of arbitrary shape." *Algorithmica*, 2(4), 403–430.
- Lumelsky, V. J. and Tiwari, S. (1994). "An algorithm for maze searching with azimuth input." *Proc., IEEE Int. Conf. on Robotic Automation*, Institute of Electrical and Electronics Engineers, New York, 111–116.
- Samet, H. (1990). *Applications of spatial data structures*, Addison-Wesley, New York.
- Smith, R. G., and Davis, R. (1981). "Framework for cooperation in distributed problem solving." *IEEE Trans. Syst. Man. Cybern.*, 11(1), 61–70.
- Warszawski, A., and Sangrey, D. (1985). "Robotics in building construction." *J. Constr. Eng. Manage.*, 111(3), 260–280.